

Memmy

让所有 AI 记住同一个你

跨工具记忆清零终结者：免费开源的个人 AI 记忆中枢

GitHub: [MemTensor/memmy-agent](https://github.com/MemTensor/memmy-agent) • MIT • ~2k★

开源项目类

本地优先 SQLite

四层记忆

Windows 实测跑通

2026-10-03

目录 = 你的 14 个疑问

1. Memmy 是什么？"记忆清零"是什么痛？
2. 和每个 Agent 自带的记忆文件有何本质区别？
3. 架构三件套怎么分工？
4. 四层记忆 L1/L2/L3/Skill 各存什么？
5. 记忆是怎么写进去的？为何会"摘要排队"？
6. 记忆为什么要打分？（奖励系统）
7. 检索为什么用 4 通道族而不是纯向量？
8. 候选怎么融合、排序、终筛？
9. 怎么接入 Claude Code / Cursor？
10. L2 策略怎么结晶成 Skill？
11. 隐私和模型依赖怎么回事？
12. 和 mem0 / MemGPT 比定位差在哪？
13. 这项目给我什么能力？
14. 动手实验：5 步跑通记忆闭环（Windows 实测）

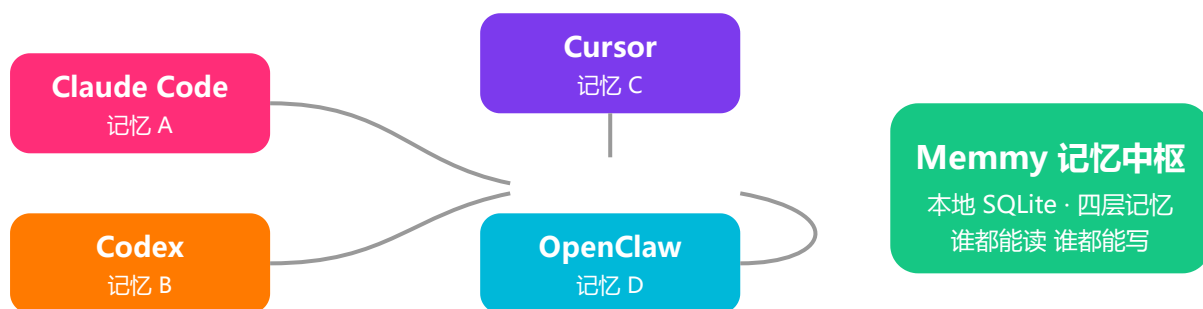
Q1

Memmy 是什么？"跨工具记忆清零"是什么痛？

一句话：Memmy 是一个跑在你自己电脑上的"个人记忆中枢"，让 Claude Code、Codex、Cursor 等所有 AI Agent 共享同一份长期记忆——口号是 "All AI remember the same you"。

场景故事：你周一在 Claude Code 里调通了 deeptutor，踩了 4 个坑、定下了"用 python -m 启动"的结论；周二换 Codex 继续做，它对你的历史一无所知——项目目标、踩过的坑、为什么改了方案，全部清零。你只能手动粘贴摘要，或者干脆从头再来。这就是"跨工具记忆清零"。

Memmy 的解法分三步：Remember（把本机各 Agent 的协作历史自动整理成结构化记忆）→ Relay（换工具时带着项目背景、偏好、进度接着干）→ Act（它自己也是个 Agent，能整理资料、合并方案、续跑任务）。



各自为政的记忆孤岛 → 收敛成一份共享的个人上下文层

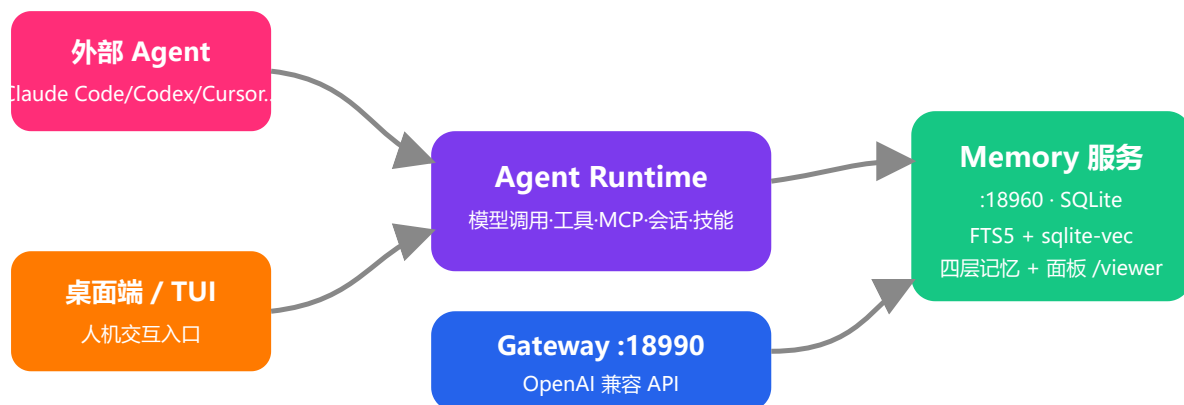
Q2

和每个 Agent 自带的记忆文件（CLAUDE.md、Cursor Rules）有何本质区别？

维度	各 Agent 自带记忆	Memmy
归属	每个工具一份孤岛（CLAUDE.md / ~/.codex / state.vscdb...）	全工具共用一个库 ，一份记忆到处可用
写入	手动维护，靠人自觉	Hook 自动抓取每轮对话（turn.complete），也支持历史全量扫描导入
结构	平铺文本，越写越长	四层结构化记忆（轨迹→策略→世界模型→技能），逐层自动进化
检索	全文塞进上下文或干脆不检索	4 通道混合检索 + 打分排序，只注入相关的 10 条左右
遗忘/纠错	没有机制	价值打分 + 时间衰减 + 归档（gain<-0.05 的 L2 会被归档）

类比：自带记忆是**每个人自己的笔记本**；Memmy 是**团队共享的知识库 + 专职图书管理员**——管理员不只收藏，还会提炼摘要、总结规矩、写成操作手册。

Q3 架构三件套怎么分工？



所有入口最终都读写同一个本地 Memory 服务（实验证实：不装桌面端、纯 CLI 也能全功能跑）

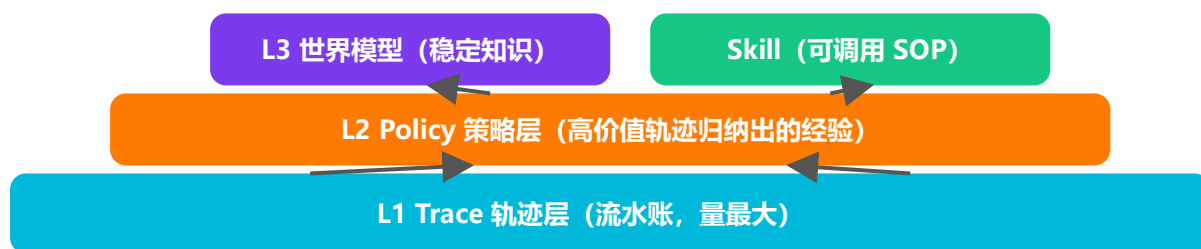
- **Memory 服务** (Fastify + SQLite, 默认 `127.0.0.1:18960`) : 唯一的记忆存取点, 向量、全文、检索、进化流水线全在这。
- **Agent Runtime**: Memmy 自己当 Agent 时的"大脑"——消息循环、工具、MCP、技能。
- **Gateway** (`:18990`) : 把 Runtime 包装成 OpenAI 兼容 API, 别的客户端也能接。

Q4 四层记忆 L1/L2/L3/Skill 各存什么？

这是 Memmy 最核心的设计：**记忆不是一堆聊天记录，而是一个会自动“蒸馏升值”的四层体系。**

层	存什么	怎么产生	类比
L1 Trace 轨迹	原始对话：用户请求、Agent 回复、工具调用、错误签名	turn.complete 自动抓取 / 手动 add / 历史扫描	监控流水账
L2 Policy 策略	从多条高价值 L1 里归纳出的 程序性经验 ：触发条件、步骤、边界、避坑指南	奖励达标后由 LLM 归纳 (I2Induction)	经验总结帖
L3 World Model 世界模型	关于项目/环境/约束的 稳定事实知识 （不是步骤）	相似 L2 聚类抽象 (I3Abstraction)	项目百科页
Skill 技能	可调用的 SOP：名字、调用指南、步骤、验证条件	合格 L2 结晶 (skill.minGain)	标准操作手册

与你已学的 [[三层记忆]] 对比 (deeptutor: L1 轨迹→L2 界面摘要→L3 用户画像)：同样是“逐层压缩”，但 Memmy 的 L2/L3 **压的不是“摘要”而是“可执行经验”**——L2 是“遇到 X 该怎么做”，L3 是“这个项目是什么样”，终点是一个能直接复用的 Skill。另外 Memmy 有显式的**奖励与遗忘**机制，deeptutor 没有。



金字塔：下层量大值杂，上层少而精；每层都有独立检索通道和注入顺序

Q5 记忆是怎么写进去的？为何会“摘要排队”？

自动链路（接入 Hook 后）：请求前 Hook 调 `turn.start`（分类意图、决定要不要回忆）→ Agent 干活 → 结束调 `turn.complete` 存原始轮次 + 逐条 L1（默认截断：用户/回复文本 4000 字、工具输出 2000 字、标签最多 8 个）。

为什么手动 add 后立刻搜不到？——实验实测的大坑：**L1 写入后进入 `summary_pending` 状态，摘要和向量化由后台异步流水线完成，完成后（`state=ready`）才可被检索。**摘要必须配 LLM；源码里 `if`

`(!llm.isConfigured()) throw SummaryModelUnconfiguredError`——**不配 LLM，记忆永远卡在排队中。**历史扫描导入的更严格：走 `summary_queued` 管道，防止半成品被注入。

摘要完成后建向量：L1 用摘要文本建 `vec_summary`，L2/L3/Skill 用 `vec`。嵌入失败进重试队列，**绝不阻塞 Agent 当前轮次。**

实验实录：不配 summary 模型时 search 永远返回 0 候选；用本地 qwen2.5:0.5b 补上并 retry 后，45 秒内“摘要→嵌入”全部完成（local MiniLM 嵌入仅 21ms）。

Q6 记忆为什么要打分？（奖励系统）

不是所有记忆都同样值钱。Memmy 借了强化学习的思路给每条 L1 打 `value`（价值）、`alpha`、`priority`（默认 0.5），决定谁能升级成 L2、谁该被遗忘：

参数	默认	含义
<code>reward.gamma</code>	0.9	位置折扣：越早的步骤权重越低（像 RL 的折扣因子）
<code>reward.lambda</code>	0.5	均匀权重与位置权重的混合比例（呼应 TD(λ)）
<code>reward.delta</code>	0.1	"力挽狂澜"加成：从上一步跑偏中救回来的步骤加分
<code>decayHalfLifeDays</code>	30	L1 优先级的时间衰减半衰期：30 天不用，重要性减半
<code>feedbackWindowSec</code>	30	回合结束后等 30 秒显式反馈，再算任务奖励回传

回合 (episode) 按话题边界/会话关闭/**2 小时空闲**合并闭合，闭合后先做反思合成与评分，奖励沿轨迹回传。这套设计与你知识库里的 [[本能学习与置信度评分]] 同类：**记忆的价值不是记的时候定的，而是被结果反复修正的。**

Q7

检索为什么用 4 通道族，而不是纯向量？

纯向量检索的弱点：对**精确标识符**（PYTHONUTF8、报错码、路径）不敏感，embedding 还可能把相近词混在一起。Memmy 用 4 族 6 通道互补：

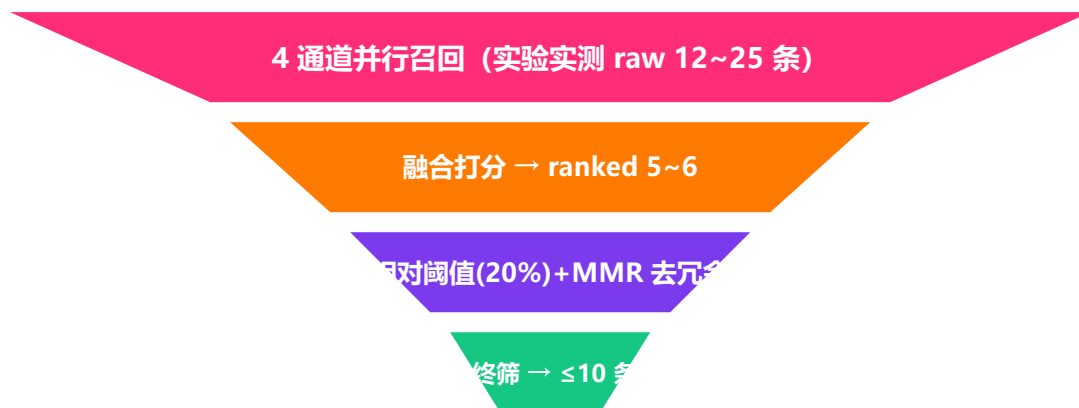
通道	技术	擅长	默认池
vec / vec_summary / vec_action	sqlite-vec (2000 行窗口内 TopK)	换词语义	Skill 12 / L1·L2 20 / L3 8
fts	SQLite FTS5 全文	词面精确命中	20/层
pattern	CJK 双字切分 + 双字符 ASCII	中文短语片段	20/层
structural	结构化片段	错误码/路径/标识符（仅 L1)	10

这与你知识库的 [[RAG检索增强生成]] 和 [[向量数据库]] 互补：**混合检索 (hybrid search)** 在“语义泛化”和“字面精确”之间取兼得。中文做了特殊照顾——查询会先切出两字 pattern。

另外有个**意图门**：任务类请求才全层召回；“你还记得以前什么”不查 L3；闲聊和元指令直接跳过检索（省时省钱）。**turn_start** 还会排除**当前会话**的 L1，防止刚说过的话又从长期记忆里绕回来。

Q8

候选怎么融合、排序、终筛？（漏斗全解）



实验真实漏斗：raw 24 → ranked 6 → 阈值砍 18 → LLM 终筛留 4

融合公式（每个候选取最好通道分，再加层质量加成）：

```
relevance = max(通道分)
            + min(weightPriority,0.3)×value×时间衰减    # L1
            + skillEtaBlend×eta                          # Skill
            + 0.2×clamp01(gain)                          # L2
            + 0.4×Σ 1/(60+通道名次+1)                   # RRF 跨通道共识加成
```

- **RRF** (Reciprocal Rank Fusion, 倒数排名融合)：一条记忆被**多个通道同时命中**就加分——“语义说相关 + 字面也对上”的更可信。
- **MMR** (最大边际相关)：按 **0.7×相关性 - 0.3×冗余** 选结果，避免注入 10 条几乎一样的记忆。
mmrLambda 调低更多样、调高更准。
- **相对阈值**：低于最高分 20% 的直接丢；但“多通道旁路”开着——命中≥2 通道的可以幸存。
- **LLM 终筛**：机械排序后让 LLM 再做一次相关性裁判（最多留 8 条；LLM 没配/挂了就保留机械 top6，**不会因此失忆**）。

实验实录（重要坑）：用 qwen2.5:0.5b 做终筛模型，搜精确词 "PYTHONUTF8" 时目标 L1 被小模型**误杀** (kept 2 / dropped 3)。终筛模型太弱会乱杀正确记忆——实战要么配强模型，要么 `retrieval.llmFilterEnabled: false`。

最终注入按 **L1 → L2 → L3 → Skill → 决策指导** 顺序渲染，普通片段 ≤640 字、Skill 只注入名字+200 字简介，且明确标注“这是历史记忆，须对照当前请求核实”——**记忆是证据，不是指令**（防提示注入的细节）。

Q9 怎么接入 Claude Code / Cursor 这些 Agent?

两条路，且“扫历史”和“实时联动”是两回事：

接入档位	机制	能干什么
历史扫描	直接读各 Agent 本地历史文件（如 Claude Code 的 <code>~/.claude/projects/**/*.jsonl</code> ）	把已有对话一次性导入成 L1（增量同步 + 水位去重；一条完整对话轮 = 一条 L1）
Hook / 插件	往 Agent 里装钩子（Claude Code Hook、Cursor Hook、OpenCode 原生插件）	新对话自动 <code>turn.start/complete</code> 、自动召回注入、任务续跑
Skill	写入记忆指令文件，让 Agent 主动调 <code>memmy-memory</code> CLI	按需读写记忆（WorkBuddy/Pi/qwenwork 用这档）

内置 9 个来源：Cursor、Claude Code、Codex、OpenCode、OpenClaw、Hermes、WorkBuddy、Pi、qwenwork；还能手动添加自定义 Agent（会先验证本机真的装了它，防止张冠李戴）。接入命令就一条：

```
memmy-memory init --agent claude,codex。
```

实验实录：本实验库启动后 `autoScanKnownAgents` 默认开启，自动把本机 Claude Code 的 54 条真实历史轨迹 + 1 个 `learn-project Skill` 导入了 SQLite——之后用换词查询“之前中文环境下 Python 报错乱码怎么解决的”，同时命中了手动写入的 GBK 坑和自动导入的 `deeptutor` PATH 坑。这就是推文说的“把跨工具清零的记忆收成一句能问的话”。

Q10 L2 策略怎么结晶成 Skill?

Skill = 从合格 L2 里“结晶”出的可调用 SOP，有名字、调用指南、步骤、验证条件。门槛参数：

参数	默认	作用
<code>skill.minSupport</code>	1	至少几条证据支持（防一条经验就立规矩）
<code>skill.minGain</code>	0.02	该策略带来的收益增益下限（要“真的有用”）
<code>skill.minEtaForRetrieval</code>	0.1	eta（预期效率）低于它就不参与召回

同理 L2 归纳要 `minTraceValue 0.005` + 与已有 L2 相似度 0.65 归并 + `gain ≥ 0.02` 激活；`gain` 掉到 -0.05 以下会被归档（软遗忘）。L2 聚类成 L3 要簇内相似度 ≥ 0.3 ，L3 置信度 ≥ 0.2 才可召回。每一层晋升都是“有证据、有收益、可撤销”的，还有 Dream 机制（手动/定时记忆整理，带日志和回滚）。

Q11 隐私和模型依赖怎么回事？

- **数据全在本地**：SQLite 存 `~/.memmy/memory-service/memory.sqlite`，服务只绑 127.0.0.1，提供导出/一键清除。
- **模型三种玩法**：① 注册送 token（云端体验）；② **BYOK** 自带 key（15+ 供应商，OpenAI 兼容端点也行）；③ **全本地**：**embedding 用 `provider: local` (`transformers.js` + `all-MiniLM-L6-v2`, 首次自动下载模型)**，**摘要/进化 LLM 指到本地 Ollama**——实验正是这么跑的，数据零出机。
- 六个模型槽独立配置：主模型、Embedding、记忆摘要、技能进化、ASR、画图。

Q12 和 mem0 / MemGPT(Letta) 比定位差在哪？

方案	定位	差异点
Memmy	个人级、跨工具、本地优先的 记忆基础设施	卖点是"接入你已有的 9 个 Agent 生态"，四层自动进化 + 奖励遗忘，带桌面端/TUI
mem0	给 开发者 的记忆 API/中间件	嵌进你自己写的应用里用；不负责扫描你本机的其他 Agent
MemGPT/Letta	一个 自带记忆 OS 的 Agent 框架	记忆围绕它自己的 Agent 运行；跨工具共享不是主目标

一句话：**mem0 是给应用装记忆，Letta 是造一个记得住事的 Agent，Memmy 是给你的"全部 AI 工具"装同一份记忆**。与你已学的 [[三层记忆]] (deeptutor, 单应用内个性化) 相比，Memmy 把同一思想推广到了"操作系统的个人上下文层"。

Q13 这个项目给我什么能力？

1. **终结换工具清零**：Claude Code ↔ Codex ↔ Cursor 接力干活，"帮我把这个项目找回来"一句话恢复目标、判断演变、产物、下一步。
2. **个人偏好跟随**："中文 Windows 先设 PYTHONUTF8"、"PDF 彩色鲜艳"这类规矩只教一次，所有工具都记得。
3. **坑库自动沉淀**：踩过的错（错误签名进 structural 通道）以后搜报错码就能翻出旧解法。
4. **经验变 SOP**：重复做对的事自动结晶成 Skill，之后一键复用（还能被 learn-project 这类 skill 引用）。
5. **当本地记忆后端**：自己写 Agent 时，把它当现成的记忆服务用——HTTP API + CLI + OpenAI 兼容网关齐全。
6. **审计与调参**：/viewer 面板能看每次检索的完整漏斗（谁被召回、谁被 LLM 砍了），记忆可查可删可导出。

Q14

动手实验：5 步跑通"写入→摘要→嵌入→语义找回" (Windows 实测全通过)

主实验：全本地记忆闭环（无 systemd、无云端 API）

环境：Windows 11 + Node v24 + 本地 Ollama (qwen2.5:0.5b 做摘要) + Memmy local embedding。完整命令与真实输出见 `exercise/experiment-log.md`。

Step 1 构建并启动本地记忆服务

```
git clone https://github.com/MemTensor/memmy-agent.git && cd memmy-agent
npm install && npm run memory:build
node Memory/dist/src/server/index.js --config ../exercise/config.yaml --port 18960
```

目的：不装桌面端，纯源码跑最小记忆服务。✅ 验证：`GET /api/v1/health` 返回 `"ok":true`，`memoryLayers= ["L1","L2","L3","Skill"]`，`fullText=fts5`、`vector=native`。

Step 2 手动写入 L1 记忆

```
node Memory/dist/src/cli/index.js add "img2threejs 项目的坑：中文 Windows 默认 GBK 编码，跑 Python 前必须先 setx PYTHONUTF8 1" --token local-token
```

✅ 返回 `stored:1, trace_47b1bd...`。⚠️ 关键发现：此时 `state=summary_pending`，**不配摘要 LLM 就永远搜不到**（源码 L362 直接抛错）。

Step 3 配置本地摘要模型并重试流水线

```
# config.yaml: summary/evolution → openai_compatible http://127.0.0.1:11434/v1 qwen2.5:0.5b
memmy-memory reload-config
curl -X POST .../api/v1/memory/<trace_id>/processing/retry
```

✅ 日志：`[capture.summarize]` 成功 → `[embedding]` `provider=local, durationMs=21` → `state:"ready"`。qwen 生成的真实摘要：「在 Windows 上设置正确的编码，以支持 Python 测试...」

Step 4 换词语义检索（零词重叠也能找回）

```
memmy-memory search "之前中文环境下Python 运行报错乱码怎么解决的" --verbose
```

查询词与记忆零重叠（报错/乱码 vs GBK/假失败）仍命中；还一并召回自动扫描导入的本机 Claude Code 真实轨迹。漏斗日志：`raw 24` → `ranked 6` → 阈值砍 `18` → 终筛留 `4`。

Step 5 跨层写入 L2 策略并验证分层召回

```
memmy-memory add "Windows 中文系统跑 Python 前先 setx PYTHONUTF8 1, 能预防 90% 编码假失败" --layer L2
memmy-memory search "编码假失败怎么预防" --verbose
```

✓ hits 排序: Skill(1.08) → **L2 policy(0.66)** → UserMemory(0.53) → L1 trace(0.26)——四层同场竞技、按质量加分排序。

附加实验：终筛模型的坑 + 三大环境坑

B. 小模型终筛误杀: search "PYTHONUTF8" (精确标识符), 机械排序明明有目标 L1, qwen2.5:0.5b 终筛却把它砍了。解法: `llmFilterEnabled: false` 或换强模型。

C1. config 格式坑: README/官方文档里 `profiles/activeProfile` 的示例在 v1.2.0 已是 legacy, 启动直接报错——要按 `Memory/src/config/index.ts` 的扁平格式写 (summary/evolution/embedding 平铺在 memmyMemory 下)。

C2. ollama pull 被代理打断: fake-ip DNS 触发 Ollama "redirect target not allowed" 保护, CLI 挂代理无效。旁路: curl 走代理直下 GGUF + `ollama create` 本地导入。

C3. Windows 无 systemd: 官方一键脚本面向 Linux; Windows 用源码构建 + node 直启即可, 功能不打折。